Name: Simon Offenberger S2410306027

Gruppe: 1 ~~2~~

Punkte: / 24

I2C - TEMPERATURSENSOR

AUFGABE

Programmieren Sie die Ansteuerung des I2C-Temperatursensors MCP9803 am APUS-Board. Die aktuelle Temperatur (auf 0,5°C genau) soll zyklisch auf dem Display angezeigt bzw. aktualisiert werden. Programmieren Sie zusätzlich TSET und THYST Werte (Active-low, Comparator-Mode) nach Ihren Vorstellungen und steuern Sie mit dem Alert-Output (per Interrupt) eine beliebige LED an.

Erstellen Sie dazu einen eigenen Treiber (.c und .h). Folgende Funktionen soll der Treiber minimal unterstützen:

- Temperatursensor initialisieren (und damit auch I2C) und konfigurieren (z.B. Alert)
- Temperaturwert auslesen
- Schwellwerte TSET und THYST setzen

HINWEISE

- Der I2C-Treiber muss zum Teil noch finalisiert werden.
- Achten Sie darauf, die korrekten PINs für I2C zu initialisieren (SDA und SCL), da es hierbei mehrere Möglichkeiten gibt.
- Der MCP9803 hat ein I2C-Timeout von 35ms, daher kann der Code nicht per Single-Step debugged werden.
- Aktivieren Sie I2C in der Datei stm32f0xx_conf.h und inkludieren Sie stm32f0xx_i2c.c

FRAGEN

- Wofür wird FAULT QUEUE verwendet?
- Wie viele Temperatursensoren können parallel maximal am I2C-Bus betrieben werden?
- Welche Auflösung des Sensors ist nach dem Reset eingestellt?
- Wozu kann der One-Shot Mode verwendet werden?

ABGABE

- Sourcecode (16 Punkte)
- Beantwortete Fragen (4 Punkte)
- Funktionsbeschreibung der Software und ein Foto des Displays im Betrieb (4 Punkte)

© Jürgen Nußbaummüller

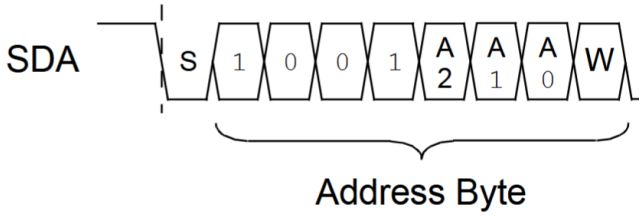
1 Fragen

1.1 Wofür wird FAULT QUEUE verwendet?

Die *Fault Queue* dient als Filter, um Fehlauslösungen des `ALERT`-Pins zu reduzieren. Dazu muss die gemessene Temperatur T_A für eine einstellbare Anzahl aufeinanderfolgender Wandlungszyklen oberhalb von T_{SET} liegen, bevor `ALERT` aktiviert wird. Entsprechend muss T_A auch für die eingestellte Anzahl von Zyklen unterhalb von T_{HYST} liegen, bevor `ALERT` wieder deaktiviert wird bzw. ein neuer Interrupt ausgelöst werden kann.

1.2 Wie viele Temperatursensoren können parallel maximal am I2C-Bus betrieben werden?

Am I^2C-Bus können maximal **acht** MCP9800/1/2/3-Sensoren parallel betrieben werden, da es **3** adressierbare Pins (A0, A1, A2) gibt, die jeweils mit GND oder VDD verbunden werden können. Dadurch ergeben sich $2^3 = 8$ mögliche Adresskombinationen für die Sensoren.



1.3 Welche Auflösung des Sensors ist nach dem Reset eingestellt?

Nach dem Reset bzw. nach Power-up ist standardmäßig eine Auflösung von **9 Bit** eingestellt. Das entspricht einer Temperaturauflösung von **0,5 °C**.

bit 5-6	ΣΔ **ADC RESOLUTION** bits
	00 = 9 bit or 0.5°C (Power-up default)
	01 = 10 bit or 0.25°C
	10 = 11 bit or 0.125°C
	11 = 12 bit or 0.0625°C

1.4 Wozu kann der One-Shot Mode verwendet werden?

Der *One-Shot Mode* wird für **stromsparende Anwendungen** verwendet. In diesem Modus führt der Sensor auf Anforderung genau **eine einzelne Temperaturmessung** durch und kehrt danach automatisch wieder in den *Shutdown Mode* zurück. Das ist besonders nützlich, wenn die Temperatur nur gelegentlich und gezielt durch einen Controller abgefragt werden soll.

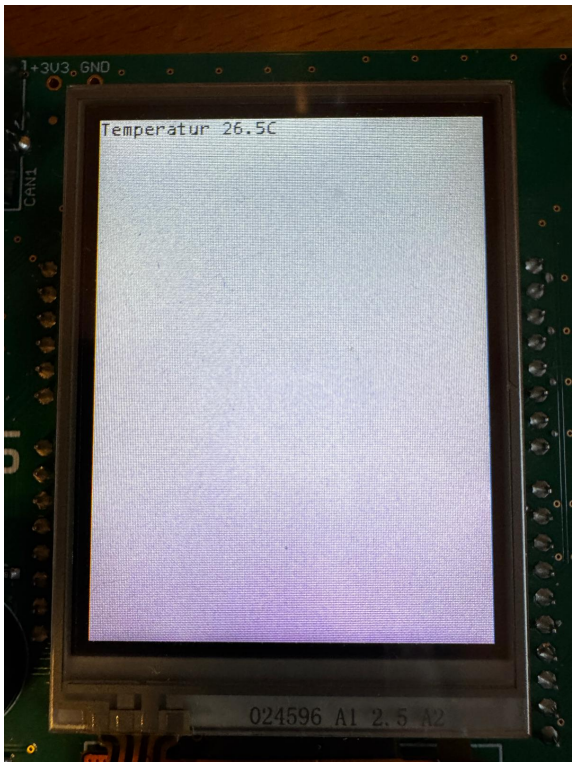
2 Funktionsbeschreibung

Im Projekt wird die aktuelle Temperatur über I2C vom MCP9800 Sensor ausgelesen und auf einem LCD-Display angezeigt. Zusätzlich wird ein Alarm ausgelöst, wenn die gemessene Temperatur den eingestellten Schwellenwert überschreitet. Der `ALERT`-Pin des Sensors ist mit einem externen Interrupt verbunden, der bei Aktivierung eine LED einschaltet, um den Alarm visuell anzuzeigen. Die Temperaturwerte werden kontinuierlich aktualisiert. Bei aktivem Alarm (Temperatur über T_{SET}) leuchtet die LED4 auf, und bei Rückkehr unter T_{HYST} erlischt sie wieder.

2.1 MCP Driver

Der Treiber für den MCP9800 Temperatursensor baut auf den I2C Treiber auf. Dieser ist so ausgelegt, dass mehrere Sensoren parallel am I2C-Bus betrieben werden können. Dazu muss für jeden Sensor ein eigener Handle angelegt werden. Im Handle werden im Bereich **init** der Initialisierungszustand des MCP9800 gesetzt und anschließend mit der Funktion `MCP9800_Temp_Init` initialisiert. Bei jedem Aufruf einer Treiber Funktion ist der entsprechende Handle zu übergeben. Status der Funktion wird über den Rückgabewert der Funktion zurückgegeben. Hier wird die I2C_RetType verwendet, um Fehler zu erkennen und entsprechend zu reagieren.

2.2 Bild vom Display



3 Code

3.1 main.c

```c
/**
  ******************************************************************************
  * @file    main.c
  * @author  Simon Offenberger
  * @version V1.0
  * @date    14-October-2025
  * @brief   This file Contains the main Function which calles the current poject main
  ******************************************************************************
  */

/* Includes ------------------------------------------------------------------*/
#include "stm32f0xx_conf.h"
#include "sysdelay.h"
#include "3bit_cnt.h"
#include "alarm.h"
#include "LedTimer.h"
#include "Interrupt_Proj.h"
#include "Uart_Vcom.h"
#include "Uart_DMA.h"
#include "ADC_Temp_prj.h"
#include "i2c_temp.h"

/* Private typedef -----------------------------------------------------------*/
/* Private define ------------------------------------------------------------*/
/* Private macro -------------------------------------------------------------*/
/* Private variables ---------------------------------------------------------*/
/* Private function prototypes -----------------------------------------------*/
/* Private functions ---------------------------------------------------------*/

/**
  * @brief  Initialize the sys tick timer (M0 core) which is used for delay.
  * @param  None
  * @retval None
  */
void SysTick_Init(void)
{
    /* init the sys tick timer to be called every 1ms */
    SysTick_Config(SystemCoreClock / 1000);
}

/**
  * @brief  This is the SysTick interrupt handler which is called every 1ms.
           We have to increment the HAL tick counter which is used for SysDelay
  * @param  None
  * @retval None
  */
void SysTick_Handler()
{
    SysDelay_IncTicks();
}

/**
* @brief  Main program
* @param  None
* @retval None
*/
int main(void)
{
    /* init the sys tick counter (needed for SysDelay) */
    SysTick_Init();

    // switch to project I2C Temperature Sensor
    main_I2C_Temp();
}
```

3.2 i2c_temp.h

```c
/**
 * @file I2C_Temp_prj.h
 * @author Simon Offenberger
 * @brief I2C Temperature Sensor Project which indicates a Alert via LED4 and shows the current temperature on the LCD.
 * @version 1.0
 * @date 2026-03-09
 *
 * @copyright Copyright (c) 2026
 *
 */

#ifndef __MAIN_I2C_TEMP_H
#define __MAIN_I2C_TEMP_H

#include <stdint.h>

/** @brief Temperature polling interval in milliseconds. */
#define I2C_POLL_INTERVAL_MS 250

/** @brief Text buffer size for LCD output. */
#define LCD_TEXT_BUFFER_SIZE 32

/** @brief Alert threshold temperature in degree Celsius. */
#define TEMP_ALERT_THRESHOLD 28.5f
/** @brief Alert hysteresis temperature in degree Celsius. */
#define TEMP_ALERT_HYSTERESIS 27.0f

/**
 * @brief Main function for the Project I2C Temperature Sensor.
 */
void main_I2C_Temp(void);

#endif /* __MAIN_I2C_TEMP_H */
```

3.3 i2c_temp.c

```c
/**
 * @file I2C_Temp_prj.c
 * @author Simon Offenberger
 * @brief I2C Temperature Sensor Project which indicates a Alert via LED4 and shows the current temperature on the LCD.
 * @version 1.0
 * @date 2026-03-09
 *
 * @copyright Copyright (c) 2026
 *
 */

#include "board_led.h"
#include "sysdelay.h"
#include "i2c_temp.h"
#include "board_MCP_temp.h"
#include "Lcd.h"
#include <stdlib.h>
#include <stdio.h>

// includes for STM32F0xx Standard Peripheral Library
#include "stm32f0xx.h"
#include "stm32f0xx_gpio.h"
#include "stm32f0xx_exti.h"
#include "stm32f0xx_syscfg.h"
#include "stm32f0xx_rcc.h"

/**
 * @brief  Configures EXTI line 4 for MCP9800 alert interrupt on PD4.
 */
void Alert_Initialize_EXTI(void)
{
  // Enable peripheral clocks
  // Peripherals are not accessible until their clocks are enabled.
  // SYSCFG is needed to map EXTI lines to GPIO pins !
  RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG, ENABLE);
  RCC_AHBPeriphClockCmd(RCC_AHBENR_GPIODEN, ENABLE);

  GPIO_InitTypeDef initStruct;

  // Load default values into structure
  GPIO_StructInit(&initStruct);

  // Configure alert input pin for external interrupt
  initStruct.GPIO_Pin = GPIO_Pin_4;
  initStruct.GPIO_Mode = GPIO_Mode_IN;
  initStruct.GPIO_PuPd = GPIO_PuPd_UP; // Alert pin is active low, so we use pull-up (external PU also populated)
  GPIO_Init(GPIOD, &initStruct);

  // Connect EXTI lines to GPIO pins
  SYSCFG_EXTILineConfig(EXTI_PortSourceGPIOD, EXTI_PinSource4); // EXTI4 -> PD4

  // Configure EXTI lines
  EXTI_InitTypeDef extiInitStruct;
  extiInitStruct.EXTI_Line = EXTI_Line4;
  extiInitStruct.EXTI_Mode = EXTI_Mode_Interrupt;
  extiInitStruct.EXTI_Trigger = EXTI_Trigger_Rising_Falling; // Interrupt on both edge (alert asserted)
  extiInitStruct.EXTI_LineCmd = ENABLE;
  EXTI_Init(&extiInitStruct);

  // Enable interrupt request in NVIC
  NVIC_EnableIRQ(EXTI4_15_IRQn);
}

/**
 * @brief  Handles EXTI Line4 interrupt generated by MCP9800 alert.
 */
void EXTI4_15_IRQHandler(void)
{
  // Check if EXTI Line4 has a pending interrupt.
  if (EXTI->PR & EXTI_PR_PR4)
  {
    // Clear the EXTI Line4 pending bit no read modify write!!!
    // Because it would reset another pending flag as well!
    // 1 written to the pending register clears the pending bit
    EXTI->PR = EXTI_PR_PR4;

    // Set LED to signal Alert
    if(GPIO_ReadInputDataBit(GPIOD, GPIO_Pin_4) == Bit_RESET){
      LED_On(LED4);
    }
    else{
      LED_Off(LED4);
    }
  }
}

/**
 * @brief Main function for the Project I2C Temperature Sensor Exercise 8.
 */
void main_I2C_Temp()
{
  // Enable global interrupts
  __enable_irq();

  // Initialize Peripheral
  LED_Initialize();
  Lcd_Initialize();
  Lcd_Fill(0, 0, LCD_WIDTH, LCD_HEIGHT, RGB565_WHITE);

  // Initialize MCP9800 temperature sensor with user-selected configuration.
```

```c
  const MCP9800_Init_StructTypeDef mcp9800Init = {
      .Shutdown_Mode = MCP9800_SHUTDOWN_MODE_DISABLE,
      .Alert_Mode = MCP9800_COMPARATOR_MODE,
      .Alert_Polarity = MCP9800_ALERT_POLARITY_ACTIVE_LOW,
      .Fault_Queue = MCP9800_FAULT_QUEUE_1,
      .Resolution = MCP9800_ADC_RESOLUTION_0_5C,
      .One_Shot = MCP9800_ONE_SHOT_DISABLE};

  MCP9800_Temp_HandleTypeDef MCP9800_Handle = {
      .i2c_handle = I2C_RTC_TEMP,
      .adr = MCP9800_SLAVE_ADDRESS,
      .init = mcp9800Init};

  MCP9800_Temp_Init(&MCP9800_Handle);

  // Program alert upper threshold and hysteresis values.
  MCP9800_Temp_Set_TSET(&MCP9800_Handle, TEMP_ALERT_THRESHOLD);
  MCP9800_Temp_Set_THYST(&MCP9800_Handle, TEMP_ALERT_HYSTERESIS);

  // Initialize EXTI for Alert pin to signal overtemperature events via LED4.
  Alert_Initialize_EXTI();

  while (1)
  {
    float temp = 0;

    // Periodically poll sensor temperature via I2C.
    MCP9800_Temp_ReadTemp(&MCP9800_Handle, &temp);

    char temp_str[LCD_TEXT_BUFFER_SIZE] = {0};

    // Update current temperature on LCD.
    snprintf(temp_str, LCD_TEXT_BUFFER_SIZE, "Temperatur␣%3.1fC", temp);

    Lcd_Print(0, 0, temp_str, RGB565_BLACK, RGB565_WHITE, &Consolas);

    SysDelay_Delay(I2C_POLL_INTERVAL_MS);
  }
}
```

4 Module

4.1 board_i2c.h

```c
/**
 ******************************************************************************
 * @file    i2c.h
 * @author  Andreas Oyrer
 * @version V1.0
 * @date    10-March-2022
 * @brief   APUS Board I2C control Header File.
 ******************************************************************************
 */

#ifndef __I2C_H
#define __I2C_H

/* Includes ------------------------------------------------------------------*/
#include <stdint.h>
#include "stm32f0xx_i2c.h"

/* Exported types ------------------------------------------------------------*/
#define I2C_RTC_TEMP I2C1

typedef enum
{
  I2C_OK = 0,
  I2C_ERR_NACK,
  I2C_ERR_SLAVE_ADDRESS,
  I2C_ERR_INVALID_PARAM,
  I2C_ERR_TIMEOUT
} I2C_RetType;

/* Exported functions ------------------------------------------------------- */

/**
 * @brief  Initialize I2C including GPIO ports.
 * @param  I2C_Handle Pointer to I2C peripheral instance.
 * @retval The result of the operation.
 */
I2C_RetType I2C_Initialize(I2C_TypeDef *I2C_Handle);

/**
 * @brief  Write data to a i2c slave device.
 * @param  I2C_Handle Pointer to I2C peripheral instance.
 * @param  slave_address Address of the I2C slave device.
 * @param  sendData Data to be sent to the I2C slave device.
 * @param  bytesToSend Number of bytes to send to the I2C slave device.
 * @retval The result of the operation.
 */
I2C_RetType I2C_Write(I2C_TypeDef *I2C_Handle, const uint16_t slave_address, const uint8_t *sendData, const uint8_t
     bytesToSend);

/**
 * @brief  First writes data to a i2c slave device and then reads back the content.
 *         Therefore the first i2c frame is sent without a STOP bit.
 *         The read transfer starts with a repeated START condition.
 * @param  I2C_Handle Pointer to I2C peripheral instance.
 * @param  slave_address Address of the I2C slave device.
 * @param  sendData Data to be sent to the I2C slave device.
 * @param  bytesToSend Number of bytes to send to the I2C slave device.
 * @param  recData Pointer to store received data from the I2C slave device.
 * @param  bytesToRead Number of bytes to read from the I2C slave device.
 * @retval The result of the operation.
 */
I2C_RetType I2C_WriteRead(I2C_TypeDef *I2C_Handle, const uint16_t slave_address, const uint8_t *sendData, const uint8_t
     bytesToSend, uint8_t *recData, const uint8_t bytesToRead);

#endif
```

4.2 board_i2c.c

```c
/**
 ******************************************************************************
 * @file    i2c.c
 * @author  Andreas Oyrer
 * @version V1.0
 * @date    10-March-2022
 * @brief   APUS Board I2C control Source File.
 ******************************************************************************
 */

/* Includes ------------------------------------------------------------------*/
#include "stm32f0xx_gpio.h"
#include "stm32f0xx_rcc.h"
#include "i2c.h"
#include "sysdelay.h"

/* Private typedef -----------------------------------------------------------*/
/* Private define ------------------------------------------------------------*/
#define I2C_TIMEOUT 100

/* Private macro -------------------------------------------------------------*/
/* Private variables ---------------------------------------------------------*/
/* Private function prototypes -----------------------------------------------*/
static int32_t I2C_WaitForFlagOrNackToBeSet(I2C_TypeDef *I2C_Handle, uint32_t flag);

/**
 * @brief  Initialize I2C including GPIO ports.
 * @param  I2C_Handle Pointer to I2C peripheral instance.
 * @retval The result of the operation.
 */
I2C_RetType I2C_Initialize(I2C_TypeDef *I2C_Handle)
{
    if(I2C_Handle == 0)
    {
        return I2C_ERR_INVALID_PARAM;
    }

    I2C_InitTypeDef initStructI2C;
    GPIO_InitTypeDef initStructGPIO;

    if (I2C_Handle == I2C_RTC_TEMP)
    {
        /* GPIO Ports Clock Enable */
        RCC_AHBPeriphClockCmd(RCC_AHBENR_GPIOBEN, ENABLE);

        GPIO_StructInit(&initStructGPIO);

        /* Configure GPIO pins for I2C: PB6 PB7 */
        initStructGPIO.GPIO_Pin = GPIO_Pin_6 | GPIO_Pin_7;
        initStructGPIO.GPIO_Mode = GPIO_Mode_AF;
        GPIO_Init(GPIOB, &initStructGPIO);
        GPIO_PinAFConfig(GPIOB, GPIO_PinSource6, GPIO_AF_1);
        GPIO_PinAFConfig(GPIOB, GPIO_PinSource7, GPIO_AF_1);

        /* init I2C1 */
        RCC_APB1PeriphClockCmd(RCC_APB1Periph_I2C1, ENABLE);

        I2C_StructInit(&initStructI2C);
        /* Master-Mode, 100kHz, Analog Filter Delay = On, Rise time = 140ns, Fall time = 40ns */
        /* calculated with I2C_Timing_Configuration_V1.0.1.xls */
        initStructI2C.I2C_Timing = 0x10905F86;
        I2C_Init(I2C_Handle, &initStructI2C);
        /* enable I2C */
        I2C_Cmd(I2C_Handle, ENABLE);
    }

    return I2C_OK;
}

/**
 * @brief  Write data to a i2c slave device.
 * @param  I2C_Handle Pointer to I2C peripheral instance.
 * @param  slave_address Address of the I2C slave device.
 * @param  sendData Data to be sent to the I2C slave device.
 * @param  bytesToSend Number of bytes to send to the I2C slave device.
 * @retval The result of the operation.
 */
I2C_RetType I2C_Write(I2C_TypeDef *I2C_Handle, const uint16_t slave_address, const uint8_t *sendData, const uint8_t
    bytesToSend)
{
    uint32_t idx = 0;
    int32_t ret = 0;

    /* null pointer check */
    if (sendData == 0 || bytesToSend == 0)
    {
        return I2C_ERR_INVALID_PARAM;
    }

    // Start write transfer with AUTOEND so STOP is generated by hardware.
    I2C_TransferHandling(I2C_Handle, slave_address, bytesToSend, I2C_AutoEnd_Mode, I2C_Generate_Start_Write);

    // Wait for TXIS flag which is send when the next value can be copied to the data register
    ret = I2C_WaitForFlagOrNackToBeSet(I2C_Handle, I2C_FLAG_TXIS);

    if ((ret & I2C_FLAG_TXIS) != I2C_FLAG_TXIS)
    {
        // got NACK or timeout
        return I2C_ERR_SLAVE_ADDRESS;
    }
```

```c

    /* send data */
    for (idx = 0; idx < bytesToSend; idx++)
    {
        // send data and wait until we get a status back
        I2C_SendData(I2C_Handle, sendData[idx]);

        // Wait on TXIS only until the second to last byte.
        // After writing the last byte, TXIS is not set again.
        if ((idx + 1) < (bytesToSend))
        {
            // wait for TXIS flag which is send when the next value can be copied to the data register
            ret = I2C_WaitForFlagOrNackToBeSet(I2C_Handle, I2C_FLAG_TXIS);
            if ((ret & I2C_FLAG_TXIS) != I2C_FLAG_TXIS)
            {
                // got NACK or timeout
                return I2C_ERR_NACK;
            }
        }
    }

    return I2C_OK;
}

/**
 * @brief  First writes data to a i2c slave device and then reads back the content.
 *         Therefore the first i2c frame is sent without a STOP bit.
 *         The read transfer starts with a repeated START condition.
 * @param  I2C_Handle Pointer to I2C peripheral instance.
 * @param  slave_address Address of the I2C slave device.
 * @param  sendData Data to be sent to the I2C slave device.
 * @param  bytesToSend Number of bytes to send to the I2C slave device.
 * @param  recData Pointer to store received data from the I2C slave device.
 * @param  bytesToRead Number of bytes to read from the I2C slave device.
 * @retval The result of the operation.
 */
I2C_RetType I2C_WriteRead(I2C_TypeDef *I2C_Handle, const uint16_t slave_address, const uint8_t *sendData, const uint8_t
      bytesToSend, uint8_t *recData, const uint8_t bytesToRead)
{
    int32_t idx = 0;
    int32_t ret = 0;

    /* null pointer check */
    if (sendData == 0 || recData == 0 || bytesToRead == 0 || bytesToSend == 0)
    {
        return I2C_ERR_INVALID_PARAM;
    }

    // First frame: write register address without STOP to allow repeated START.
    I2C_TransferHandling(I2C_Handle, slave_address, bytesToSend, I2C_SoftEnd_Mode, I2C_Generate_Start_Write);

    ret = I2C_WaitForFlagOrNackToBeSet(I2C_Handle, I2C_FLAG_TXIS);

    if ((ret & I2C_FLAG_TXIS) != I2C_FLAG_TXIS)
    {
        // got NACK or timeout
        return I2C_ERR_SLAVE_ADDRESS;
    }

    /* send data */
    for (idx = 0; idx < bytesToSend; idx++)
    {
        // send data and wait until we get a status back
        I2C_SendData(I2C_Handle, sendData[idx]);

        // for Last Byte
        if ((idx + 1) == bytesToSend)
        {
            // at last byte Wait for TC Flag
            ret = I2C_WaitForFlagOrNackToBeSet(I2C_Handle, I2C_FLAG_TC);
            if ((ret & I2C_FLAG_TC) != I2C_FLAG_TC)
            {
                // got NACK or timeout
                return I2C_ERR_NACK;
            }
        }
        // only wait on TXIS until the second to last byte
        // Last byte does not get a TXIS!!
        else
        {
            // wait for TXIS flag which is send when the next value can be copied to the data register
            ret = I2C_WaitForFlagOrNackToBeSet(I2C_Handle, I2C_FLAG_TXIS);
            if ((ret & I2C_FLAG_TXIS) != I2C_FLAG_TXIS)
            {
                // got NACK or timeout
                return I2C_ERR_NACK;
            }
        }
    }

    // Second frame: repeated START for read phase, STOP auto-generated at end.
    I2C_TransferHandling(I2C_Handle, slave_address, bytesToRead, I2C_AutoEnd_Mode, I2C_Generate_Start_Read);

    /* receive data */
    for (idx = 0; idx < bytesToRead; idx++)
    {
        // for Reading wait for RXNE Flag
        ret = I2C_WaitForFlagOrNackToBeSet(I2C_Handle, I2C_FLAG_RXNE);

        if ((ret & I2C_FLAG_RXNE) != I2C_FLAG_RXNE)
        {
            // got NACK or timeout
            return I2C_ERR_NACK;
```

```
        }
        else
        {
            // received data
            recData[idx] = I2C_ReceiveData(I2C_Handle);
        }
    }

    return I2C_OK;
}

/* Private functions ---------------------------------------------------------*/
static int32_t I2C_WaitForFlagOrNackToBeSet(I2C_TypeDef *I2C_Handle, uint32_t flag)
{
    if(I2C_Handle == 0)
    {
        return I2C_ERR_INVALID_PARAM;
    }

    uint32_t initialTicks = 0;
    uint32_t isrState = 0;

    initialTicks = SysDelay_GetTicks();

    /* Poll until target flag is set, abort early on timeout or NACK. */
    do
    {
        isrState = I2C_ReadRegister(I2C_Handle, I2C_Register_ISR);
        /* check for a timeout */
        if ((SysDelay_GetTicks() - initialTicks) > I2C_TIMEOUT)
        {
            // Return 0 to indicate timeout.
            return 0x00;
        }
        if ((isrState & I2C_FLAG_NACKF) == I2C_FLAG_NACKF)
        {
            return I2C_FLAG_NACKF;
        }
    } while ((isrState & flag) != flag);

    return isrState & flag;
}
```

4.3 board_MCP_temp.h

```c
/**
 ******************************************************************************
 * @file    Board_MCP_Temp.h
 * @author  Simon Offenberger
 * @version V1.0
 * @date    9-March-2026
 * @brief   Driver for the MCP9800 Temperature Sensor on the APUS Board.
 ******************************************************************************
 */

#ifndef __BOARD_MCP_TEMP_H
#define __BOARD_MCP_TEMP_H

/* Includes ------------------------------------------------------------------*/
#include "i2c.h"

/* Defines -------------------------------------------------------------------*/
/** @brief 7-bit MCP9800 slave address shifted for STM32 SPL I2C API. */
#define MCP9800_SLAVE_ADDRESS 0x90

/** @brief MCP9800 temperature register address. */
#define MCP_REG_TEMP 0x00
/** @brief MCP9800 configuration register address. */
#define MCP_REG_CONFIG 0x01
/** @brief MCP9800 hysteresis register address. */
#define MCP_REG_HYSTERESIS 0x02
/** @brief MCP9800 limit register address. */
#define MCP_REG_LIMIT 0x03

#define ONE_SHOT_POS 7
#define ADC_RESOLUTION_POS 5
#define FAULT_QUEUE_POS 3
#define ALERT_POLARITY_POS 2
#define ALERT_MODE_POS 1
#define SHUTDOWN_MODE_POS 0

/* Exported types ------------------------------------------------------------*/
/**
 * @brief  One-shot conversion mode.
 */
typedef enum
{
  MCP9800_ONE_SHOT_DISABLE = 0,
  MCP9800_ONE_SHOT_ENABLE = 1
} MCP9800_OneShot_Enum;

/**
 * @brief  ADC resolution selection.
 */
typedef enum
{
  MCP9800_ADC_RESOLUTION_0_5C = 0x00,
  MCP9800_ADC_RESOLUTION_0_25C = 0x01,
  MCP9800_ADC_RESOLUTION_0_125C = 0x02,
  MCP9800_ADC_RESOLUTION_0_0625C = 0x03
} MCP9800_Resolution_Enum;

/**
 * @brief  Number of consecutive fault events before alert.
 */
typedef enum
{
  MCP9800_FAULT_QUEUE_1 = 0x00,
  MCP9800_FAULT_QUEUE_2 = 0x01,
  MCP9800_FAULT_QUEUE_4 = 0x02,
  MCP9800_FAULT_QUEUE_6 = 0x03
} MCP9800_FaultQueue_Enum;

/**
 * @brief  Alert output polarity.
 */
typedef enum
{
  MCP9800_ALERT_POLARITY_ACTIVE_LOW = 0,
  MCP9800_ALERT_POLARITY_ACTIVE_HIGH = 1
} MCP9800_AlertPolarity_Enum;

/**
 * @brief  Alert mode selection.
 */
typedef enum
{
  MCP9800_COMPARATOR_MODE = 0,
  MCP9800_INTERRUPT_MODE = 1
} MCP9800_AlertMode_Enum;

/**
 * @brief  Sensor shutdown mode.
 */
typedef enum
{
  MCP9800_SHUTDOWN_MODE_DISABLE = 0,
  MCP9800_SHUTDOWN_MODE_ENABLE = 1
} MCP9800_ShutdownMode_Enum;

/**
 * @brief  MCP9800 configuration bitfield values.
 */
typedef struct
{
```

```c
  MCP9800_ShutdownMode_Enum Shutdown_Mode;
  MCP9800_AlertMode_Enum Alert_Mode;
  MCP9800_AlertPolarity_Enum Alert_Polarity;
  MCP9800_FaultQueue_Enum Fault_Queue;
  MCP9800_Resolution_Enum Resolution;
  MCP9800_OneShot_Enum One_Shot;
} MCP9800_Init_StructTypeDef;

/**
 * @brief  MCP9800 driver handle.
 */
typedef struct
{
  I2C_TypeDef *i2c_handle;
  uint16_t adr;
  MCP9800_Init_StructTypeDef init;

} MCP9800_Temp_HandleTypeDef;

/* Exported functions ------------------------------------------------------- */

/**
 * @brief  Initializes the MCP9800 sensor.
 * @param  handle Pointer to MCP9800 handle structure.
 * @return I2C status.
 */
I2C_RetType MCP9800_Temp_Init(MCP9800_Temp_HandleTypeDef *handle);

/**
 * @brief  Reads the current temperature value.
 * @param  handle Pointer to MCP9800 handle structure.
 * @param  temp Pointer to destination temperature in degree Celsius.
 * @return I2C status.
 */
I2C_RetType MCP9800_Temp_ReadTemp(MCP9800_Temp_HandleTypeDef *handle, float *temp);

/**
 * @brief  Sets hysteresis temperature threshold.
 * @param  handle Pointer to MCP9800 handle structure.
 * @param  temp Hysteresis threshold in degree Celsius.
 * @return I2C status.
 */
I2C_RetType MCP9800_Temp_Set_THYST(MCP9800_Temp_HandleTypeDef *handle, float temp);

/**
 * @brief  Sets alert temperature threshold.
 * @param  handle Pointer to MCP9800 handle structure.
 * @param  temp Alert threshold in degree Celsius.
 * @return I2C status.
 */
I2C_RetType MCP9800_Temp_Set_TSET(MCP9800_Temp_HandleTypeDef *handle, float temp);

#endif
```

4.4 board_MCP_temp.c

```c
/**
 ******************************************************************************
 * @file    Board_MCP_Temp.c
 * @author  Simon Offenberger
 * @version V1.0
 * @date    9-March-2026
 * @brief   Driver for the MCP9800 Temperature Sensor on the APUS Board.
 ******************************************************************************
 */

/* Includes ------------------------------------------------------------------*/
#include "board_MCP_temp.h"

/* Private define ------------------------------------------------------------*/
#define INT_FLOAT_CONV 256.0f
#define ONLY_MSB_BIT 0x80
#define NUM_OF_TEMP_BYTES 2

/**
 * @brief  Initializes the MCP9800 sensor and writes the configuration register.
 * @param  handle Pointer to MCP9800 handle structure.
 * @return I2C status.
 */
I2C_RetType MCP9800_Temp_Init(MCP9800_Temp_HandleTypeDef *handle)
{

  if (handle == 0)
  {
    return I2C_ERR_INVALID_PARAM;
  }

  // Build MCP9800 config register from user-selected bitfields.
  const uint8_t config_reg = ((uint8_t)handle->init.Shutdown_Mode << SHUTDOWN_MODE_POS) |
                             ((uint8_t)handle->init.Alert_Mode << ALERT_MODE_POS) |
                             ((uint8_t)handle->init.Alert_Polarity << ALERT_POLARITY_POS) |
                             ((uint8_t)handle->init.Fault_Queue << FAULT_QUEUE_POS) |
                             ((uint8_t)handle->init.Resolution << ADC_RESOLUTION_POS) |
                             ((uint8_t)handle->init.One_Shot << ONE_SHOT_POS);

  I2C_RetType retval = I2C_Initialize(handle->i2c_handle);

  if (retval != I2C_OK)
  {
    return retval;
  }

  const uint8_t sendData[2] = {MCP_REG_CONFIG, config_reg};

  retval = I2C_Write(handle->i2c_handle, handle->adr, sendData, sizeof(sendData) / sizeof(sendData[0]));

  return retval;
}

/**
 * @brief  Reads the current temperature from MCP9800.
 * @param  handle Pointer to MCP9800 handle structure.
 * @param  temp Pointer to destination temperature in degree Celsius.
 * @return I2C status.
 */
I2C_RetType MCP9800_Temp_ReadTemp(MCP9800_Temp_HandleTypeDef *handle, float *temp)
{
  if (handle == 0 || temp == 0)
  {
    return I2C_ERR_INVALID_PARAM;
  }
  uint8_t recData[NUM_OF_TEMP_BYTES];
  uint8_t sendData = MCP_REG_TEMP;

  // Write temperature register address, then read back 2 bytes with repeated START.
  I2C_RetType ret = I2C_WriteRead(handle->i2c_handle, handle->adr, &sendData, 1, (uint8_t *)&recData, sizeof(recData) / sizeof
      (recData[0]));

  if (ret != I2C_OK)
  {
    return ret;
  }

  // MCP9800 temperature format is Q8.8, convert raw value to degree Celsius.
  *temp = ((float)(recData[0] << 8 | recData[1])) / INT_FLOAT_CONV;

  return ret;
}

/**
 * @brief  Sets the MCP9800 overtemperature limit (TSET).
 * @param  handle Pointer to MCP9800 handle structure.
 * @param  temp Temperature limit in degree Celsius.
 * @return I2C status.
 */
I2C_RetType MCP9800_Temp_Set_TSET(MCP9800_Temp_HandleTypeDef *handle, float temp)
{

  if (handle == 0 || temp == 0)
  {
    return I2C_ERR_INVALID_PARAM;
  }

  uint16_t temp_int = (uint16_t)(temp * INT_FLOAT_CONV);
  // THYST/TSET use MSB plus bit7 of LSB according to MCP9800 register format.
  const uint8_t sendData[3] = {MCP_REG_LIMIT, (temp_int >> 8) & 0xFF, temp_int & ONLY_MSB_BIT};
```

```c

  return I2C_Write(handle->i2c_handle, handle->adr, sendData, sizeof(sendData) / sizeof(sendData[0]));
}

/**
 * @brief  Sets the MCP9800 hysteresis temperature threshold (THYST).
 * @param  handle Pointer to MCP9800 handle structure.
 * @param  temp Hysteresis temperature in degree Celsius.
 * @return I2C status.
 */
I2C_RetType MCP9800_Temp_Set_THYST(MCP9800_Temp_HandleTypeDef *handle, float temp)
{
  if (handle == 0 || temp == 0)
  {
    return I2C_ERR_INVALID_PARAM;
  }

  uint16_t temp_int = (uint16_t)(temp * INT_FLOAT_CONV);
  // THYST/TSET use MSB plus bit7 of LSB according to MCP9800 register format.
  const uint8_t sendData[3] = {MCP_REG_HYSTERESIS, (temp_int >> 8) & 0xFF, temp_int & ONLY_MSB_BIT};

  return I2C_Write(handle->i2c_handle, handle->adr, sendData, sizeof(sendData) / sizeof(sendData[0]));
}
```